

Smart Pointers

`unique_ptr` · `shared_ptr` · `weak_ptr`

Automatic memory management without a garbage collector

The Problem with Raw Pointers



Memory Leaks

forget to call
delete



Double Delete

delete called
more than once



Dangling Ptr

use after
delete



Exception Safety

leaks when
exceptions thrown

```
// ❌ Raw pointer – who owns this? Who deletes it?  
  
int* ptr = new int(42);  
doSomething(ptr);    // throws an exception!  
delete ptr;          // ← NEVER REACHED – memory leaked! 💀  
  
// ✅ Solution: Smart pointers auto-delete when they go out of scope
```

unique_ptr

Exclusive Ownership

Only ONE owner at a time

Zero overhead vs raw pointer

Cannot be copied — only moved

Auto-deletes when out of scope

```
#include <memory>

// Create unique_ptr
auto ptr = make_unique<int>(42);    // preferred
unique_ptr<int> ptr2(new int(42));  // also valid

// Move ownership (ptr is now empty)
auto ptr3 = move(ptr);  // ptr == nullptr after this

// Automatically deleted when ptr3 goes out of scope ✓
```

Use `unique_ptr` by default — it's the right choice most of the time

Arrays

```
// unique_ptr for arrays
auto arr =
    make_unique<int[]>(5);

arr[0] = 10; // ✅ operator[]
arr[1] = 20;

// Deletes[] automatically
// when arr goes out of scope
```

Custom Deleter

```
// e.g. wrapping a C file handle
auto closeFile = [](FILE* f) {
    if (f) fclose(f);
};

unique_ptr<FILE,
    decltype(closeFile)> fp(
    fopen("data.txt", "r"),
    closeFile
);

// fclose called automatically!
```



```
// Shared ownership – reference counted
auto a = make_shared<int>(42); // ref_count = 1
auto b = a;                    // ref_count = 2
auto c = b;                    // ref_count = 3

cout << a.use_count();        // prints 3

// When ALL go out of scope → ref_count=0 → deleted ✓
```

weak_ptr observes a **shared_ptr** WITHOUT increasing the reference count. The object can be deleted even while a **weak_ptr** exists.

Use for: Breaking circular references between **shared_ptr**s, caches, observer patterns

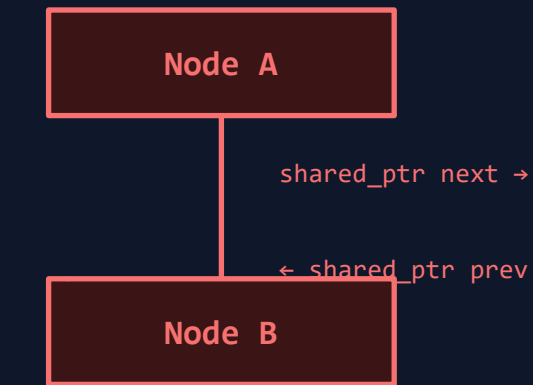
```
auto shared = make_shared<int>(42); // ref=1
weak_ptr<int> weak = shared;        // ref still 1!

// Must lock() to safely access the object
if (auto locked = weak.lock()) {    // locked is shared_ptr
    cout << *locked; // safe ✅
} else {
    cout << "Object was deleted";
}

shared.reset(); // Object deleted, ref=0
weak.expired(); // returns true ✅
```

weak_ptr::lock() returns a **shared_ptr** — empty if object is gone

Circular Reference Problem — Solved by weak_ptr



LEAK! ref_count never reaches 0

✓ Fix: use weak_ptr for back-reference

```
struct Node {  
    shared_ptr<Node> next; // owns next  
    weak_ptr<Node> prev; // observes prev  
};
```

```
// Now ref_count drops to 0 ✓  
// Nodes get cleaned up properly
```

Side-by-Side Comparison

Feature	unique_ptr	shared_ptr	weak_ptr
Ownership	Exclusive	Shared	None
Copyable	✗ No	✓ Yes	✓ Yes
Moveable	✓ Yes	✓ Yes	✓ Yes
Reference Count	✗ No	✓ Yes	✓ Yes (weak)
Can access object	✓ Direct	✓ Direct	⚠ lock() first
Overhead	● Zero	● Control block	● Control block
Use when	Default choice	Multiple owners	Break cycles / observe

When in doubt: unique_ptr. Need sharing: shared_ptr. Have cycles: weak_ptr.

Best Practices

Prefer `make_unique` / `make_shared`

```
auto p = make_unique<Foo>(); // not: unique_ptr<Foo>(new Foo())
```

Pass by raw pointer/reference when ownership NOT transferred

```
void process(Foo* raw) { ... } // callee doesn't own it
```

Return `unique_ptr` from factories

```
unique_ptr<Widget> createWidget() { return make_unique<Widget>(); }
```

Avoid `get()` unless calling C APIs

```
cFunc(ptr.get()); // ok for C APIs – don't delete the raw ptr!
```

Summary

Choose the right tool for the job

unique_ptr

DEFAULT CHOICE

One owner, zero overhead. Use for almost everything. Move to transfer.

```
#include <memory>
```

shared_ptr

MULTIPLE OWNERS

Reference counted. Use when several places genuinely share an object.

```
#include <memory>
```

weak_ptr

NON-OWNING REFERENCE

Observe a shared_ptr without owning it. Break cycles, build caches.

```
#include <memory>
```